



Trabajo práctico 1

Especificación de TADs

September 14, 2026

Algoritmos y Estructuras de Datos

INVARIANTES

Integrante	LU	Correo electrónico
Capdepont, Camila	852/25	camilacapdepont@gmail.com
Yatche, Tobias	38795612	yatchetobias@gmail.com
Davila Bustamante, Jasson Aldayr	59/22	jason.davila001@gmail.com
Brizuela, Matías	1139/24	matu.brizu@outlook.com



Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires
Ciudad Universitaria - (Pabellón I/Planta Baja)
Intendente Güiraldes 2610 - C1428EGA
Ciudad Autónoma de Buenos Aires - Rep. Argentina
Tel/Fax: (++54 +11) 4576-3300
<http://www.exactas.uba.ar>

Tipos de datos creados:

enum Categoria ES {Bronce, Plata, Oro, Platino}

IdUsuario ES $\mathbb{Z}$

Usuarios ES $\text{conj}\langle IdUsuario \rangle$

Promo ES $\text{seq}\langle \text{char} \rangle$

PromocionDatos ES $\text{struct}\langle factorPromocion : \mathbb{R}, tieneTope : \text{bool}, tope : \mathbb{R} \rangle$

Promociones ES $\text{dict}\langle Promo, PromocionDatos \rangle$

enum TipoTransaccion ES {viaje, canje, transferencia, referencia}

Transaccion ES $\text{struct}\langle ud : IdUsuario, ur : IdUsuario, tipo : TipoTransaccion, millas : \mathbb{R},$
 $distancia : \mathbb{R}, promo : Promo, factorPromocion : \mathbb{R} \rangle$

Transacciones ES $\text{seq}\langle Transaccion \rangle$

TAD AirMiles {

obs us : Usuarios

obs trans : Transacciones

obs promos : Promociones

proc crearSistema() : AirMiles {

requiere { True }

asegura {

$res.us = \{\}$ $\wedge$

$res.trans = \langle \rangle \wedge$

$esUnicaPromocion(res.promos, "NoPromo") \wedge_L$

$esPromoPorDefecto(res.promos, "NoPromo")$

}

}

pred esUnicaPromocion(Promos : Promociones, p : Promo) {

$(p \in Promos.promos) \wedge (|Promos.promos| = 1)$

}

pred esPromoPorDefecto(Promos : Promociones, p : Promo) {

$(Promos[p].factorPromocion = 1) \wedge (Promos[p].tieneTope = false) \wedge (Promos[p].tope = 0)$

}

proc registrarUsuario(inout sis : AirMiles, in u : IdUsuario) {

requiere { $sis = Sis_0 \wedge \neg(u \in sis.us)$ }

asegura {

$sis.trans = Sis_0.trans \wedge$

$sis.promos = Sis_0.promos \wedge$

$(u \in sis.us) \wedge (|sis.us| = |Sis_0.us| + 1) \wedge usuariosAntiguosPermanecenIguales(sis, Sis_0, u)$

}

}

pred usuariosAntiguosPermanecenIguales(sNew : AirMiles, sOld : AirMiles, u : IdUsuario) {

$(\forall o : IdUsuario) (o \in sNew.us \leftrightarrow (o \in sOld.us \vee o = u))$

}

```

proc registrarUsuarioConReferidor(inout sis : AirMiles, in u : IdUsuario, in ref : IdUsuario) {
  requiere { sis = Sis0 ∧ ¬(u ∈ sis.us) ∧ (ref ∈ sis.us) }
  asegura {
    sis.promos = Sis0.promos ∧
    usuariosAntiguosPermanecenIguales(sis, Sis0, u) ∧
    (u ∈ sis.us) ∧ (|sis.us| = |Sis0.us| + 1) ∧ (|sis.trans| = |Sis0.trans| + 2) ∧
    transaccionesUsuarioNuevoYReferidorEstanAgregadas(sis.trans, Sis0.trans, u, ref)
  }
}

pred transaccionesUsuarioNuevoYReferidorEstanAgregadas(tNew : Transacciones, tOld : Transacciones,
  u : IdUsuario, ref : IdUsuario) {
  todasLasTransaccionesAntiguasPermanecenIguales(tNew, tOld) ∧
  usuarioYReferidorEstanEnTransacciones(tNew, ref, 1) ∧
  usuarioYReferidorEstanEnTransacciones(tNew, u, 2)
}

pred todasLasTransaccionesAntiguasPermanecenIguales(tNew : Transacciones, tOld : Transacciones) {
  (∀ i : ℤ) (0 ≤ i < |tOld| →L tNew[i] = tOld[i])
}

[Nota: el 0, en el pred de abajo, representa al idUsuario del Sistema]
pred usuarioYReferidorEstanEnTransacciones(tNew : Transacciones, x : IdUsuario, i : ℤ) {
  (tNew[|tNew| - i].ud = 0) ∧
  (tNew[|tNew| - i].ur = x) ∧
  (tNew[|tNew| - i].tipo = Referencia) ∧
  (tNew[|tNew| - i].millas = 500)
}

proc acumularMillas(inout sis : AirMiles, in u : IdUsuario, in d : ℝ, in p : Promo) {
  requiere { sis = Sis0 ∧ (p ∈ sis.promos) ∧ (d > 0) ∧ (u ∈ sis.us) ∧ noSePasaDelTope(sis, p, u, d) }
  asegura {
    (sis.us = Sis0.us) ∧ (|sis.trans| = |Sis0.trans| + 1) ∧
    transaccionConPromoEstaAgregada(sis, Sis0.trans, u, Viaje, d,
    Sis0.promos[p].factorPromocion) ∧
    alcanzoTopeONo(sis, Sis0, p, u, d)
  }
}

pred transaccionConPromoEstaAgregada(sis : AirMiles, tOld : Transacciones,
  u : IdUsuario, t : TipoTransaccion, d : ℝ, p : Promo, f : ℝ) {
  todasLasTransaccionesAntiguasPermanecenIguales(sis.trans, tOld) ∧
  hayNuevaTransaccionConFactor(sis, u, d, f, p)
}

```

[Nota: el 0 en el pred hayNuevaTransaccionConFactor representa al ID del sistema]

```

pred hayNuevaTransaccionConFactor(sis : AirMiles, u : IdUsuario, d : ℝ, f : ℝ, p : Promo) {
  (sis.trans[sis.trans - 1].ud = 0) ∧
  (sis.trans[sis.trans - 1].ur = u) ∧
  (sis.trans[sis.trans - 1].tipo = Viaje) ∧
  (sis.trans[sis.trans - 1].millas = calculoDeMillasPorFactor(sis, d, u, f)) ∧
  (sis.trans[sis.trans - 1].distancia = d) ∧
  (sis.trans[sis.trans - 1].promo = p) ∧
  (sis.trans[sis.trans - 1].factorPromocion = f)
}

pred noSePasaDelTope(sis : AirMiles, p : Promo, u : IdUsuario, d : ℝ) {
  ¬(sis.promos[p].tieneTope) ∨L
  ((calculoDeMillasPorFactor(sis, d, u, sis.promos[p].factorPromocion)
  + calculoMillasPorPromo(sis, p)) ≤ sis.promos[p].tope)
}

pred alcanzoTopeONo(sis : AirMiles, sisOld : AirMiles, p : Promo, u : IdUsuario, d : ℝ) {
  (alcanzaTope(sisOld, p, u, d) →
  sis.promos = delKey(sisOld.promos, p)) ∨
  (sis.promos = sisOld.promos)
}

pred alcanzaTope(sisOld : AirMiles, p : Promo, u : IdUsuario, d : ℝ) {
  (sisOld.promos[p].tieneTope) ∧
  (calculoDeMillasPorFactor(sisOld, d, u, sisOld.promos[p].factorPromocion)
  + calculoMillasPorPromo(sisOld, p) = sisOld.promos[p].tope)
}

aux calculoDeMillasPorFactor(sis : AirMiles, d : ℝ, u : IdUsuario, f : ℝ) : ℝ =
  d * calculoFactorPorCategoria(sis, u) * f;

aux calculoFactorPorCategoria(sis : AirMiles, u : IdUsuario) : ℝ =
  IfthenElse(calculoMillasTotales(sis, u) ≥ 1000000, 0,
  IfthenElse(calculoMillasTotales(sis, u) ≥ 50000, 2,
  IfthenElse(calculoMillasTotales(sis, u) ≥ 10000, 1.5, 1)));

aux calculoMillasPorPromo(sis : AirMiles, p : Promo) : ℝ =
  ∑i=0|sis.trans|-1 (IfthenElse(sis.trans[i].promo = p, sis.trans[i].millas, 0));

aux calculoMillasTotales(sis : AirMiles, u : IdUsuario) : ℝ =
  ∑i=0|sis.trans|-1 (IfthenElse(sis.trans[i].ur = u, sis.trans[i].millas, 0));

```

```

proc canjearMillas(inout sis : AirMiles, in u : IdUsuario, in m :  $\mathbb{R}$ ) {
  requiere { sis = Sis0  $\wedge$  (u  $\in$  sis.us)  $\wedge$  (m > 0)  $\wedge$  (calculoMillasDisponibles(sis, u)  $\geq$  m) }
  asegura {
    sis.promos = Sis0.promos  $\wedge$ 
    sis.us = Sis0.us  $\wedge$ 

    [Nota: el 0 en el pred transaccionDeTipoEstaAgregada representa al ID del sistema]
    transaccionDeTipoEstaAgregada(sis, Sis0, u, 0, canje, m)
  }
}

pred transaccionDeTipoEstaAgregada(sis : AirMiles, Sis0 : AirMiles, uD : IdUsuario, uR : IdUsuario, tipo :
TipoTransaccion, millas :  $\mathbb{R}$ ) {
  todasLasTransaccionesAntiguasPermanecenIguales(sis, Sis0)  $\wedge$ 
  (sis.trans[|sis.trans| - 1].ud = uD)  $\wedge$ 
  (sis.trans[|sis.trans| - 1].ur = uR)  $\wedge$ 
  (sis.trans[|sis.trans| - 1].tipo = tipo)  $\wedge$ 
  (sis.trans[|sis.trans| - 1].millas = millas)
}

aux calculoMillasDisponibles(sis : AirMiles, u : IdUsuario) :  $\mathbb{R}$  =
  calculoMillasTotales(sis, u) - calculoMillasGastadas(sis, u);

aux calculoMillasGastadas(sis : AirMiles, u : IdUsuario) :  $\mathbb{R}$  =
   $\sum_{i=0}^{|sis.trans|-1} (IfthenElse(sis.trans[i].ud = u, sis.trans[i].millas, 0));$ 

proc transferirMillas(inout sis : AirMiles, in ud : IdUsuario, in ur : IdUsuario, in m :  $\mathbb{R}$ ) {
  requiere { sis = Sis0  $\wedge$  (ud  $\in$  sis.us)  $\wedge$  (ur  $\in$  sis.us)  $\wedge$  (ud  $\neq$  ur)  $\wedge$  (m > 0)  $\wedge$ 
    (calculoMillasDisponibles(sis, ud)  $\geq$  m) }
  asegura {
    sis.promos = Sis0.promos  $\wedge$ 
    sis.us = Sis0.us  $\wedge$ 
    transaccionDeTipoEstaAgregada(sis, Sis0, ud, ur, transferencia, m)
  }
}

proc consultarSaldo(in sis : AirMiles, in u : IdUsuario) :  $\mathbb{R}$  {
  requiere { u  $\in$  sis.us }
  asegura {
    res = calculoMillasDisponibles(sis, u)
  }
}

proc consultarCategoria(in sis : AirMiles, in u : IdUsuario) : Categoria {
  requiere { u  $\in$  sis.us }
  asegura {
    esAlgunaCategoria(sis, u, res)
  }
}

```

```

pred esAlgunaCategoria(sis : AirMiles, u : IdUsuario, r : Categoria) {
  ((calculoMillasTotales(sis, u) < 10000)  $\leftrightarrow$  r = Bronce)  $\wedge$ 
  (((calculoMillasTotales(sis, u)  $\geq$  10000)  $\wedge$  (calculoMillasTotales(sis, u)  $\leq$  49999))  $\leftrightarrow$  r = Plata)  $\wedge$ 
  (((calculoMillasTotales(sis, u)  $\geq$  50000)  $\wedge$  (calculoMillasTotales(sis, u)  $\leq$  999999))  $\leftrightarrow$  r = Oro)  $\wedge$ 
  (((calculoMillasTotales(sis, u)  $\geq$  1000000))  $\leftrightarrow$  r = Platino)
}

proc rankingMillas(in sis : AirMiles, in n :  $\mathbb{Z}$ ) : seq<IdUsuario> {
  requiere { (|sis.us| > 0)  $\wedge$  (n > 0) }
  asegura {
    estaOrdenadoDeMayorAMenor(sis, res)  $\wedge$ 
    ((|sis.us|  $\geq$  n)  $\rightarrow$  hayNUsuariosTop(sis, res, n))  $\wedge$ 
    (|sis.us| < n  $\rightarrow$  todosEstanEnRanking(sis, res))
  }
}

pred hayNUsuariosTop(sis : AirMiles, r : seq<IdUsuario>, n :  $\mathbb{Z}$ ) {
  |r| = n  $\wedge$ 
  ( $\forall u$  : IdUsuario) (u  $\in$  r  $\rightarrow$  u  $\in$  sis.us)  $\wedge$ 
  (( $\forall j$  : IdUsuario) (j  $\in$  sis.us  $\wedge$   $\neg$ (j  $\in$  r)  $\rightarrow_L$ 
calculoMillasDisponibles(sis, r[|r| - 1])  $\geq$  calculoMillasDisponibles(sis, j))
}

pred estaOrdenadoDeMayorAMenor(sis : AirMiles, r : seq<IdUsuario>) {
  ( $\forall i$  :  $\mathbb{Z}$ ) (0  $\leq$  i < |r| - 1  $\rightarrow_L$ 
calculoMillasDisponibles(sis, r[i])  $\geq$  calculoMillasDisponibles(sis, r[i + 1]))
}

pred todosEstanEnRanking(sis : AirMiles, r : seq<IdUsuario>) {
  ( $\forall u$  : IdUsuario) (u  $\in$  sis.us  $\leftrightarrow$  u  $\in$  r)  $\wedge$  |r| = |sis.us|
}

proc resumenTransacciones(in sis : AirMiles, in u : IdUsuario) : tupla <  $\mathbb{R}$ ,  $\mathbb{R}$ ,  $\mathbb{R}$  > {
  requiere { u  $\in$  sis.us }
  asegura {
    res0 = calculoMillaPorTipo(sis, u, viaje)  $\wedge$ 
    res1 = calculoMillaPorTipo(sis, u, canje)  $\wedge$ 
    res2 = calculoMillaPorTipo(sis, u, transferencia)
  }
}

aux calculoMillaPorTipo(sis : AirMiles, u : IdUsuario, t : TipoTransaccion) :  $\mathbb{R}$  =
   $\sum_{i=0}^{|sis.trans|-1}$  (IfthenElse(haceTransaccionDeTipoT(sis, u, t, i), sis.trans[i].millas, 0));

pred haceTransaccionDeTipoT(sis : AirMiles, u : IdUsuario, t : TipoTransaccion, i :  $\mathbb{Z}$ ) {
  (sis.trans[i].tipo = t)  $\wedge$  ((sis.trans[i].ud = u)  $\vee$  (sis.trans[i].ur = u))
}

```

```

proc iniciarPromocion(in sis : AirMiles, in p : Promo, in f :  $\mathbb{R}$ ) {
  requiere { sis = Sis0  $\wedge$   $\neg(p \in sis.promos) \wedge f \geq 1$  }
  asegura {
    sis.us = Sis0.us  $\wedge$ 
    sis.trans = Sis0.trans  $\wedge$ 
    |sis.promos| = |Sis0.promos| + 1  $\wedge$ 
    (p  $\in$  sis.promos)  $\wedge$ 
    promoRegistrada(sis.promos, p, f, false, -1)  $\wedge$ 
    [Nota: Cuando el parámetro tope es igual a -1 es porque la promoción no tiene tope.]
    otrasPromosPermanecenIguales(sis.promos, Sis0.promos, p)
  }
}

pred promoRegistrada(promos : Promociones, p : Promocion, f :  $\mathbb{R}$ , tieneTope : bool, tope :  $\mathbb{R}$ ) {
  (promos[p].factorPromocion = f)  $\wedge$  (promos[p].tieneTope = tieneTope)  $\wedge$  (promos[p].tope = tope)
}

pred otrasPromosPermanecenIguales(pNew : Promociones, pOld : Promociones, p : Promo) {
  ( $\forall o$  : Promo) (o  $\in$  pOld  $\wedge$  o  $\neq$  p  $\rightarrow_L$  pNew[o] = pOld[o])
}

proc iniciarPromocionConTope(in sis : AirMiles, in p : Promo, in f :  $\mathbb{R}$ , in tope :  $\mathbb{R}$ ) {
  requiere { sis = Sis0  $\wedge$   $\neg(p \in sis.promos) \wedge (f \geq 1) \wedge (tope > 0)$  }
  asegura {
    sis.us = Sis0.us  $\wedge$ 
    sis.trans = Sis0.trans  $\wedge$ 
    |sis.promos| = |Sis0.promos| + 1  $\wedge$ 
    (p  $\in$  sis.promos)  $\wedge$ 
    promoRegistrada(sis.promos, p, f, true, tope)  $\wedge$ 
    otrasPromosPermanecenIguales(sis.promos, Sis0.promos, p)
  }
}

proc finalizarPromocion(in sis : AirMiles, in p : Promo) {
  requiere { sis = Sis0  $\wedge$  (p  $\in$  sis.promos)  $\wedge$  p  $\neq$  "NoPromo" }
  asegura {
    sis.us = Sis0.us  $\wedge$ 
    sis.trans = Sis0.trans  $\wedge$ 
    |sis.promos| = |Sis0.promos| - 1  $\wedge$ 
     $\neg(p \in sis.promos) \wedge$ 
    otrasPromosPermanecenIguales(sis.promos, Sis0.promos, p)
  }
}

proc elQueTieneMasMillas(in sis : AirMiles) : IdUsuario {
  requiere { |sis.us| > 0 }
  asegura {
    (res  $\in$  sis.us)  $\wedge$  esMaximo(sis, res)
  }
}

pred esMaximo(sis : AirMiles, r : IdUsuario) {
  ( $\forall u$  : IdUsuario) (u  $\in$  sis.us  $\wedge$  u  $\neq$  r  $\rightarrow$  calculoMillasTotales(sis, r)  $\geq$  calculoMillasTotales(sis, u))
}

```

```

proc elMasCanjeador(in sis : AirMiles) : IdUsuario {
  requiere { |sis.trans| > 0 }
  asegura {
    res ∈ sis.us ∧
    (∀u : IdUsuario) (u ∈ sis.us →
      (sumaMillasCanjeadasYTransferidas(sis, res) ≥ sumaMillasCanjeadasYTransferidas(sis, u)))
  }
}

aux sumaMillasCanjeadasYTransferidas(sis : AirMiles, u : IdUsuario) : IdUsuario =
  ∑i=0|sis.trans|-1 (IfthenElse(sis.trans[i].ud = u ∧ sis.trans[i].tipo = Canje,
    sis.trans[i].millas, 0)) +
  ∑i=0|sis.trans|-1 (IfthenElse(sis.trans[i].ud = u ∧ sis.trans[i].tipo = Transferencia,
    sis.trans[i].millas, 0));

proc paresTransferidoresExclusivos(in sis : AirMiles) : seq⟨tupla < IdUsuario, IdUsuario >⟩ {
  requiere { |sis.us| > 1 }
  asegura {
    sonUsuariosExclusivos(sis, res) ∧ noHayDuplasRepetidas(res)
  }
}

pred sonUsuariosExclusivos(sis : AirMiles, r : seq⟨tupla < IdUsuario, IdUsuario >⟩) {
  (∀t : tupla < IdUsuario, IdUsuario >) (t ∈ res ↔ t0 ≠ t1 ∧L
    seTransfirieronAlMenosUnaVez(sis, t0, t1) ∧L esExclusivo(sis, t0, t1)))
}

pred seTransfirieronAlmenosUnaVez(sis : AirMiles, t0 : IdUsuario, t1 : IdUsuario) {
  (∃i : ℤ) (0 ≤ i < |sis.trans| ∧ (sis.trans[i].tipo = Transferencia) ∧L
    ((t0 = sis.trans[i].ud ∧ t1 = sis.trans[i].ur) ∨ (t0 = sis.trans[i].ur ∧ t1 = sis.trans[i].ud)))
}

pred esExclusivo(sis : AirMiles, t1 : IdUsuario, t2 : IdUsuario) {
  (∀i : ℤ) (0 ≤ i < |sis.trans| ∧L (sis.trans[i].tipo = Transferencia) →L
    ((t0 = sis.trans[i].ud ∨ t0 = sis.trans[i].ur) ↔ (t1 = sis.trans[i].ud ∨ t1 = sis.trans[i].ur)))
}

pred noHayDuplasRepetidas(res : seq⟨tupla < IdUsuario, IdUsuario >⟩) {
  (∀i, j : ℤ) ((0 ≤ i < |res| ∧ 0 ≤ j < |res| ∧ i ≠ j) →L res[i] ≠ res[j] ∧ res[i]0 ≠ res[j]1)
}

```